Proof of Partial Correctness and Safety Properties of `soc-agent`

Waleed Akrabi

March 27, 2026

1 Statement of Scope

This document proves the *partial correctness*, *logical consistency*, and *fail-closed safety properties* of the implemented `soc-agent` control plane. It does *not* prove universal empirical detection accuracy against all real-world security incidents, because such a claim would require omniscient ground truth over an open world with external APIs, incomplete evidence, and non-deterministic model behavior.

Definition 1 (Control-plane correctness)**.** For this document, `soc-agent` is *control-plane correct* if the following hold:

(1) the investigation task graph actually executed is a deterministic function of the normalized alert;

(2) the scheduler never executes a task before its dependencies are satisfied;

(3) retries, timeouts, and early-stop behavior are bounded and fail closed;

(4) adapter execution is never attempted on the remediation path unless the explicit policy conditions for execution are satisfied;

(5) persisted memory, correlation, and replay operations preserve the stored incident semantics;

(6) the stated invariants compose without contradiction in `run_investigation`.

2 Assumptions

Assumption 1 (Runtime semantics)**.** Python, SQLite/Postgres transactions, JSON parsing, and the filesystem behave according to their documented semantics.

Assumption 2 (Adapter contract)**.** External model providers and integration adapters either return values conforming to the interfaces used by `soc-agent` or raise exceptions.

Assumption 3 (Deduplicated entities on the application path)**.** On the main application path, the entity lists written into the memory store are deduplicated. This assumption is satisfied by the implemented path

$$\texttt{extract_entities_from_graph} \to \texttt{MemoryStore.write_memory},$$

because the extractor accumulates entities in sets and emits sorted unique lists.

Remark 1. Assumption 3 is needed only for the exact baseline-count monotonicity claim. The main safety claims do not depend on it.

3 Boundary of What Cannot Be Proved

Proposition 1 (Universal detection accuracy is not provable here)**.** *No proof from the present source tree can establish that* `soc-agent` *is always factually accurate about whether an arbitrary real-world alert is malicious, benign, or complete in evidentiary coverage.*

Proof. The system depends on external evidence sources, operator configuration, and language-model outputs. Ground-truth maliciousness is not derivable from the code alone. Therefore the strong claim "the system is universally accurate" is not a theorem of this implementation. □

Remark 2. This limitation does not weaken the formal proof below. It means only that the provable statement must be about implemented semantics and safety, not omniscient real-world truth.

4 Planner and Commander

Lemma 1 (Planner totality and determinism)**.** *For any input alert object,* `Planner.build_plan(alert)` *returns a finite plan whose* `plan_id`*, alert*

type, objective, task identities, dependencies, retry bounds, and timeouts are deterministic functions of the alert fields used by the planner.

Proof. The planner normalizes only the alert type and severity, computes a UUIDv5 from a fixed namespace string and alert metadata, chooses one of finitely many hard-coded task templates, and materializes tasks from that template. No random source, network input, clock value, or mutable global state participates. Therefore the output is deterministic. The plan is finite because every branch returns a list of exactly five task specifications. □

Lemma 2 (Planner-generated dependency graph is acyclic)**.** *Every plan emitted by* `Planner.build_plan` *is a directed acyclic graph.*

Proof. Let the canonical order of task roles be

$$(\texttt{recon}, \texttt{threat_intel}, \texttt{forensics}, \texttt{remediation}, \texttt{reporter}).$$

In every planner branch:

(a) `recon` has no dependencies;

(b) `threat_intel` depends only on `recon`;

(c) `forensics` depends only on `recon`;

(d) `remediation` depends only on `threat_intel` and `forensics`;

(e) `reporter` depends only on earlier tasks.

Hence every edge points from an earlier position in the above order to a later one. A directed cycle would require some edge to point backwards in that order, which never occurs. Therefore the graph is acyclic. □

Lemma 3 (Commander does not let the LLM choose the task graph)**.** *The actual executed task graph is independent of the commander's LLM output.*

Proof. Inside `Commander.investigate`, the LLM response is parsed into a local variable `plan` used only for logging the objective text. The actual execution path is

`Commander._run_plan(alert, agents_map)` → `self.planner.build_plan(alert)`.

Therefore the scheduled plan topology comes from the deterministic planner, not from the LLM. If the LLM call fails or returns invalid JSON, the commander falls back to a fixed local dictionary for logging and still invokes the deterministic planner. Hence the executed DAG is LLM-independent. □

5 Scheduler

Lemma 4 (Dependency safety)**.** *The scheduler never starts a task unless all of its dependencies are either completed or skipped.*

Proof. At each iteration the ready set is defined exactly by

```
all(dep in completed or dep in skipped for dep in task.dependencies).
```

Only tasks in that ready set are passed to `asyncio.gather` for execution. Therefore a task cannot start before every dependency is marked completed or skipped. □

Lemma 5 (Blocked propagation)**.** *If any dependency of a remaining task has status failed or blocked, then that task is marked blocked and is not executed.*

Proof. At the start of every loop, `_block_unreachable_tasks` scans remaining tasks and checks whether any dependency belongs to the failed or blocked sets. Every such task is immediately assigned status `blocked`, written into `task_results`, added to the blocked set, and removed from the remaining set. Since it is removed before the ready-set computation, it cannot be executed afterward. □

Lemma 6 (Bounded retries and timeout fail-closed)**.** *For each task with retry bound r, the scheduler invokes the runner at most $r + 1$ times. If the runner exceeds its timeout or raises an exception on all attempts, the task finishes in status `failed`.*

Proof. The scheduler increments the local counter `attempts` once per loop of `_run_task`. After each failed attempt it checks whether `attempts > max_retries`. The first time this inequality holds, it returns a failed `TaskExecutionResult`. Hence the number of attempts is at most $r + 1$. Timeout exceptions are caught and converted to a failure string; arbitrary exceptions are also caught and stored as the last error. Therefore timeout and repeated exception cases fail closed. □

Lemma 7 (Early stop is optional-only)**.** *Early stop can skip only optional tasks, and only after every mandatory non-reporter task has completed with average confidence at least the configured threshold, with no failed or blocked tasks present.*

Proof. The predicate `_should_early_stop` first rejects early stop if any non-optional task other than `reporter` is not completed. It then computes the

mandatory confidence and rejects early stop when the confidence is undefined or below threshold. It also rejects early stop if any task has failed or been blocked. If the predicate is true, the caller marks only the currently remaining tasks with `task.optional = True` as skipped. No mandatory task is skipped by this branch. □

Lemma 8 (Termination on planner-generated plans)**.** *For every planner-generated plan, `Scheduler.run` terminates and returns a finite `ScheduleResult`.*

Proof. By the acyclicity lemma, every nonempty remaining set contains at least one task whose dependencies are satisfied or one task made blocked by failed predecessors. In each outer-loop iteration, at least one task is removed from the finite set `remaining`: either by blocking, by execution completion/failure, or by early-stop skipping. Since the initial number of tasks is finite, the loop terminates after finitely many removals. □

6 Remediation Safety

Proposition 2 (Only well-formed proposals reach the executor)**.** *The remediation executor is called only for objects that were successfully materialized as `ActionProposal` values.*

Proof. The remediation agent invokes the executor only inside the loop `for proposal in proposals`. That list is produced only after `validate_action_proposals(raw)` returns successfully. If parsing or validation does not produce such proposals, the loop body does not run and the executor is not called. □

Proposition 3 (Exact authorization condition)**.** *For an `ActionExecutionRequest` q, `ExecutionPolicy.decide(q)` yields `should_execute = True` if and only if all four conditions hold:*

(i) execution policy is enabled;

(ii) the action type is allowlisted;

(iii) the adapter supports the action type;

(iv) the request has `allow_execution = True`.

Proof. Inspect the decision tree in `ExecutionPolicy.decide`. The function returns `should_execute = False` immediately whenever any one of the four listed conditions is false, and returns `should_execute = True` only in the final branch reached exactly when all four are true. Therefore the condition is necessary and sufficient. □

Corollary 1 (No unauthorized remediation execution attempt)**.** *On the implemented remediation path, no adapter execution is attempted unless the policy is enabled, the action is allowlisted, the adapter supports it, and auto-execution was explicitly enabled.*

Proof. The remediation path is

$$\texttt{RemediationAgent} \rightarrow \texttt{ActionExecutorTool.run} \rightarrow \texttt{adapter.execute}.$$

The tool calls `adapter.execute` only in the branch where the policy decision has `should_execute = True`. By the previous proposition, this happens exactly under the four authorization conditions. Moreover, if no explicit policy object is supplied, the remediation agent constructs one with `allowed_actions = ()`, so execution still fails closed by default. Hence no unauthorized adapter execution attempt can occur on this path. □

Remark 3. This is a code-level authorization theorem. It does not assert that an authorized external API call must succeed remotely; it asserts that the code does not even attempt adapter execution before the authorization predicate is satisfied.

7 Memory, Correlation, and Replay

Lemma 9 (At most one stored memory per run identifier)**.** *The memory store contains at most one `IncidentMemory` row per `run_id`.*

Proof. The `incident_memory` schema declares `run_id` unique. The write path performs an `INSERT ... ON CONFLICT(run_id) DO UPDATE`. Therefore distinct rows with the same `run_id` cannot coexist. □

Lemma 10 (Successful memory write is post-checked)**.** *If `MemoryStore.write_memory` returns normally with value m, then m is retrievable from the store by its `run_id`.*

Proof. After issuing the insert or upsert, `write_memory` immediately calls `get_memory_by_run_id(record.run_id)`. If that query returns `None`, the method raises `RuntimeError`. Therefore any normal return is accompanied by a successful postcondition check. □

Lemma 11 (Baseline monotonicity on the application path)**.** *Under Assumption 3, each completed application-path memory write increments the observed baseline count by exactly one for each unique extracted entity in that memory.*

Proof. The application path obtains entities from `extract_entities_from_graph`, which emits unique lists. Then `MemoryStore._update_baselines` iterates over each entity value once and either inserts a new observed baseline with incident count 1 or updates the existing row by adding 1. Because no duplicate entity value appears within a bucket on the application path, the increment occurs exactly once per unique extracted entity. □

Lemma 12 (Correlation returns only matching prior context)**.** *Every incident returned by `CorrelationService.get_prior_context` matches at least one entity extracted from the queried alert, and no run identifier is returned twice.*

Proof. For each extracted entity bucket, the correlation service queries `list_memories_for_entity(can value)`. That store method includes a memory only if the queried value appears in one of the stored entity lists under the same canonical entity type. The correlation service tracks a `seen_runs` set and skips any memory whose `run_id` has already been emitted. Therefore every returned incident matches at least one queried entity and duplicate runs are eliminated. □

Lemma 13 (Replay preserves stored alert semantics)**.** *If a run identifier exists in memory, `replay_investigation(run_id, ...)` reconstructs the stored alert from `alert_json` and passes that reconstructed alert to `run_investigation`.*

Proof. `replay_investigation` first retrieves the memory by `run_id`. It then calls `_reconstruct_alert(memory.alert_json)` and passes the resulting alert object directly to `run_investigation`. If the run identifier is missing, it raises `ValueError` instead of inventing a replacement alert. Therefore replay preserves the persisted alert semantics or fails explicitly. □

8 Main Theorem

Theorem 1 (Implemented control-plane correctness of `soc-agent`)**.** *Under Assumptions 1–3, for every call to*

`run_investigation(config, alert, dry_run, ...)`

that returns normally, the following statements are true:

(1) the task DAG actually executed is a deterministic function of the input alert and not of the commander's LLM output;

(2) every task executed by the scheduler begins only after its dependencies are completed or skipped;

(3) *the scheduler terminates in finitely many steps on the planner-generated plan and assigns each scheduled task exactly one final result among completed, failed, blocked, or skipped;*

(4) *adapter execution is attempted on the remediation path if and only if the explicit authorization conditions of the execution policy are satisfied;*

(5) *if memory is enabled, every normal return from* `write_memory` *implies that the run's incident memory is persisted and retrievable by* `run_id`*;*

(6) *if a replay is later requested for that persisted* `run_id`*, the replay path reconstructs the stored alert semantics instead of manufacturing a different alert.*

Proof. Item (1) follows from the planner determinism lemma and the commander LLM-independence lemma. Item (2) follows from the scheduler dependency-safety lemma. Item (3) follows from the blocked propagation, bounded-retry, early-stop, and termination lemmas. Item (4) follows from the exact authorization proposition and its corollary. Item (5) follows from the memory post-check lemma. Item (6) follows from the replay lemma. The statements are composable because the application pipeline wires these subsystems in series without any contradictory branch that bypasses the proved guards. Therefore the theorem holds. □

9 Empirical Validation of the Proof Obligations

The proofs above are deductive. The current workspace also provides empirical validation that the implemented code conforms to the proved obligations.

On March 27, 2026, the command

```
pytest soc-agent/tests -q
```

completed with

```
225 passed, 8 skipped.
```

The passing tests include, among others:

(1) planner determinism and expected DAG structure;

(2) scheduler concurrency, retry bounds, timeout handling, and optional-only early stop;

(3) execution-policy gating and adapter routing;

(4) memory-store persistence, baseline updates, and correlation behavior;

(5) entity extraction and replay reconstruction;

(6) approval identity, approval queue, worker queue, and application-level investigation flow.

Remark 4. The tests do not replace the proof. They provide implementation evidence that the source under inspection matches the logical properties proved above.

10 Conclusion

The strongest defensible statement is the following:

> `soc-agent` is mathematically and logically correct with respect to its implemented control-plane semantics: deterministic planning, dependency-safe scheduling, bounded failure behavior, fail-closed remediation authorization, persistent incident memory, correlation, and replay. What is *not* proved is universal empirical threat-detection accuracy over arbitrary real-world incidents.

That statement is both rigorous and true for the code inspected here.